



Cấu trúc dữ liệu & giải thuật

Data Structure and Algorithms

Chương 2: ĐỆ QUY

TS. Võ Văn Tuấn Dũng

Trường Đại học Tài nguyên Môi trường TP. HCM

2.1. DẪN NHẬP

● Cho $S(n) = 1 + 2 + 3 + \dots + n$

$\Rightarrow S(10)? S(11)?$

$$S(10) = 1 + 2 + \dots + 10 = 55$$

$$\begin{aligned} S(11) &= 1 + 2 + \dots + 10 + 11 = 66 \\ &= S(10) + 11 \\ &= 55 + 11 = 66 \end{aligned}$$

2.1. DẪN NHẬP

Bước 2. Thế ngược

- Xác định kết quả bài toán đồng dạng từ đơn giản đến phức tạp → Kết quả cuối cùng.

$$S(n) = S(n-1) + n$$

$$S(n-1) = S(n-2) + n-1$$

$$\dots = \dots + \dots$$

$$S(1) = S(0) + 1$$

$$S(0) = 0$$

Bước 1. Phân tích

- Phân tích thành bài toán đồng dạng nhưng đơn giản hơn.
- Dừng lại ở bài toán đồng dạng đơn giản nhất có thể xác định ngay kết quả.

2.2. KHÁI NIỆM ĐỆ QUY



❖ Khái niệm

Vấn đề đệ quy là vấn đề được định nghĩa bằng chính nó.

❖ Ví dụ

Tổng $S(n)$ được tính thông qua tổng $S(n-1)$.

❖ 2 điều kiện quan trọng

- Tồn tại bước đệ quy.
- Điều kiện dừng.

2.2. KHÁI NIỆM ĐỆ QUI

Trong thực tế ta thường gặp những đối tượng bao gồm chính nó hoặc được định nghĩa dưới dạng của chính nó. Ta nói các đối tượng đó được xác định một cách đệ quy.

Một khái niệm đệ quy gồm 2 thành phần:

- Thành phần neo (dừng) (anchor)
- Thành phần đệ quy

➡ ***Phần đệ quy thể hiện tính “quy nạp”***

➡ ***Phần neo đảm bảo cho tính dừng.***

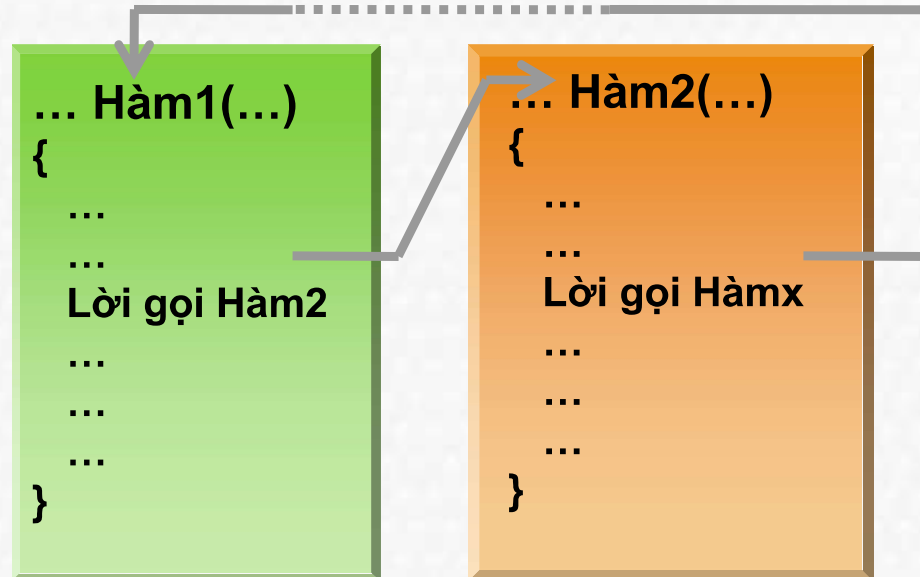
2.3. LẬP TRÌNH ĐỆ QUY

2.3.1. Hàm đệ quy (Recursive Functions)

- Một hàm được gọi là đệ quy nếu bên trong thân của hàm đó có lời gọi hàm lại chính nó một cách trực tiếp hay gián tiếp.



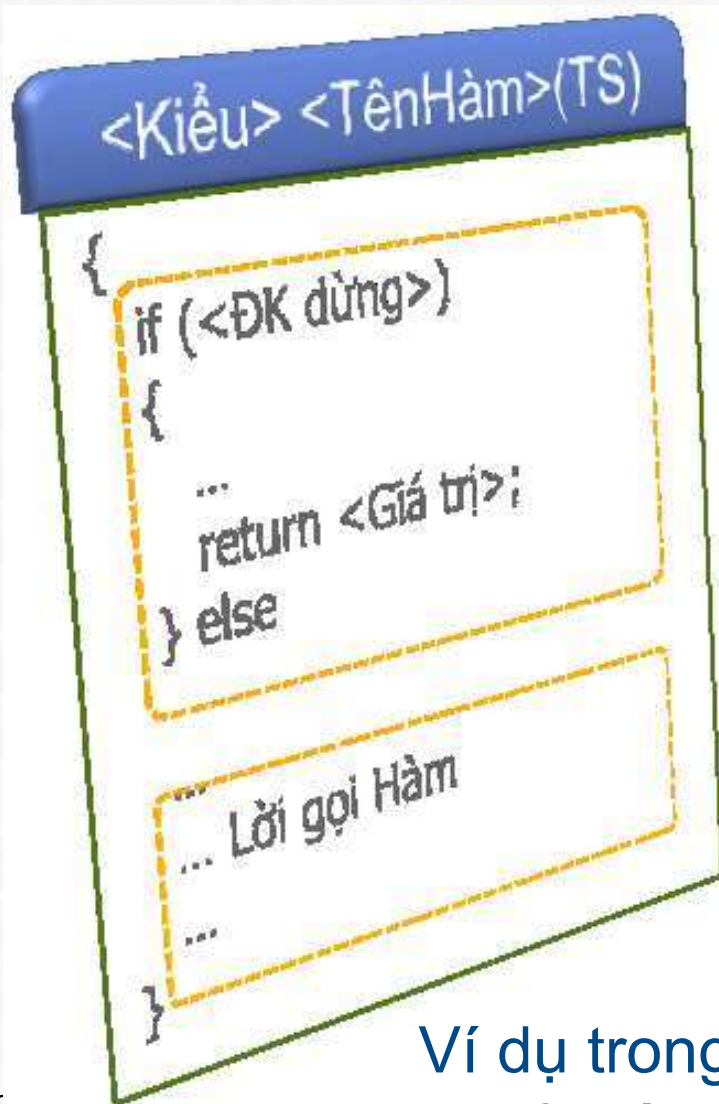
Đệ quy trực tiếp



Đệ quy gián tiếp

2.3. LẬP TRÌNH ĐỆ QUY

2.3.2. Cấu trúc hàm đệ quy



Phần cơ bản (Base step)

- Phần khởi đầu tính toán hoặc điểm kết thúc của thuật toán
- Không chứa phần đang được định nghĩa

Phần đệ quy (Recursion step)

- Có sử dụng thuật toán đang được định nghĩa.

Ví dụ trong hàm giai thừa; phần cơ bản là $n! = 1$, nếu $n = 0$; và phần đệ quy là $n! = n(n-1)!$ nếu $n > 0$.

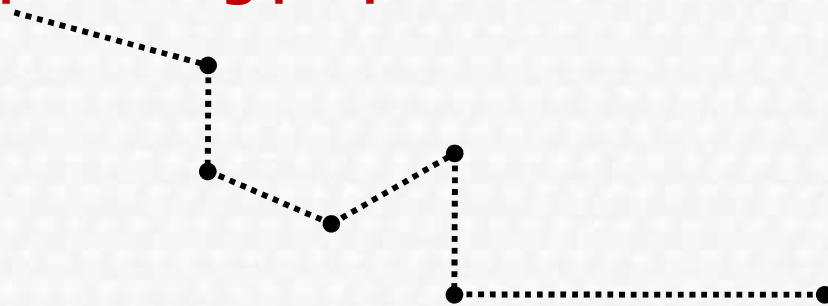
2.3. LẬP TRÌNH ĐỆ QUI

2.3.3. Phân loại



2.3. LẬP TRÌNH ĐỆ QUY

- **Đệ quy tuyến tính:** Trong thân hàm có duy nhất một lời gọi hàm gọi lại chính nó.



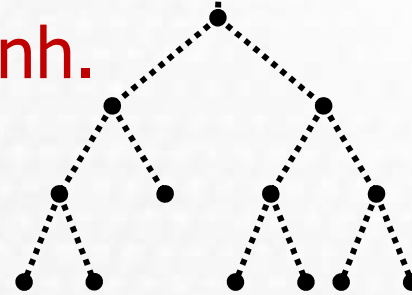
```
<Kiểu> TênHàm (<TS>) {  
    if (<ĐK dừng>) {  
        ...  
        return <Giá Trị>;  
    } else  
        ... TênHàm(<TS>); ...  
}
```

[Đệ quy tuyến tính.pptx](#)

2.3. LẬP TRÌNH ĐỆ QUI



Đệ quy nhị phân: Trong thân hàm có hai lời gọi hàm gọi lại chính nó một cách tường minh.

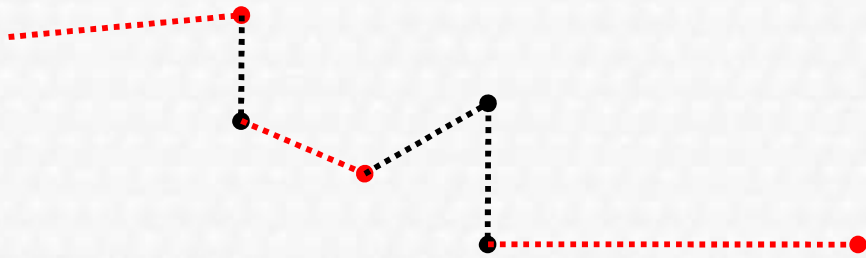


[Đệ quy nhị phân.pptx](#)

```
<Kiểu> TênHàm (<TS>) {  
    if (<ĐK dừng>) {  
        ...  
        return <Giá Trị>;  
    } else  
        ... TênHàm(<TS>;  
        ... TênHàm(<TS>;  
        ...  
}
```

2.3. LẬP TRÌNH ĐỆ QUI

Đệ quy hỗ tương: Trong thân hàm này có lời gọi hàm tới hàm kia và bên trong thân hàm kia có lời gọi hàm tới hàm này. Điều kiện dừng và trả về của 2 hàm có thể giống hoặc khác nhau.

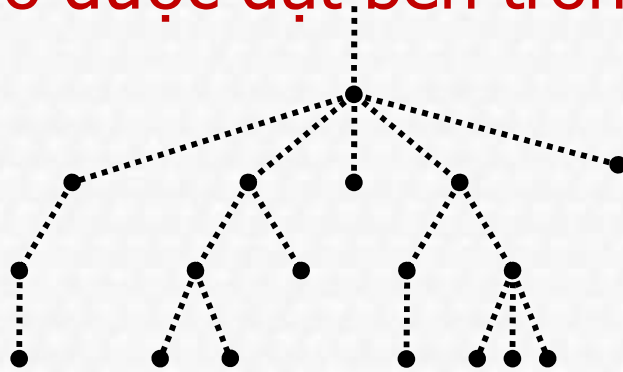


```
<Kiểu> TênHàm1(<TS>) {  
    if (<ĐK dừng>)  
        return <Giá trị>;  
    ... TênHàm2(<TS>); ...  
}  
  
<Kiểu> TênHàm2(<TS>) {  
    if (<ĐK dừng>)  
        return <Giá trị>;  
    ... TênHàm1(<TS>); ...  
}
```

Đệ quy hỗ tương.pptx

2.3. LẬP TRÌNH ĐỆ QUI

- **Đệ quy phi tuyến:** Trong thân hàm có lời gọi hàm lại chính nó được đặt bên trong thân vòng lặp.

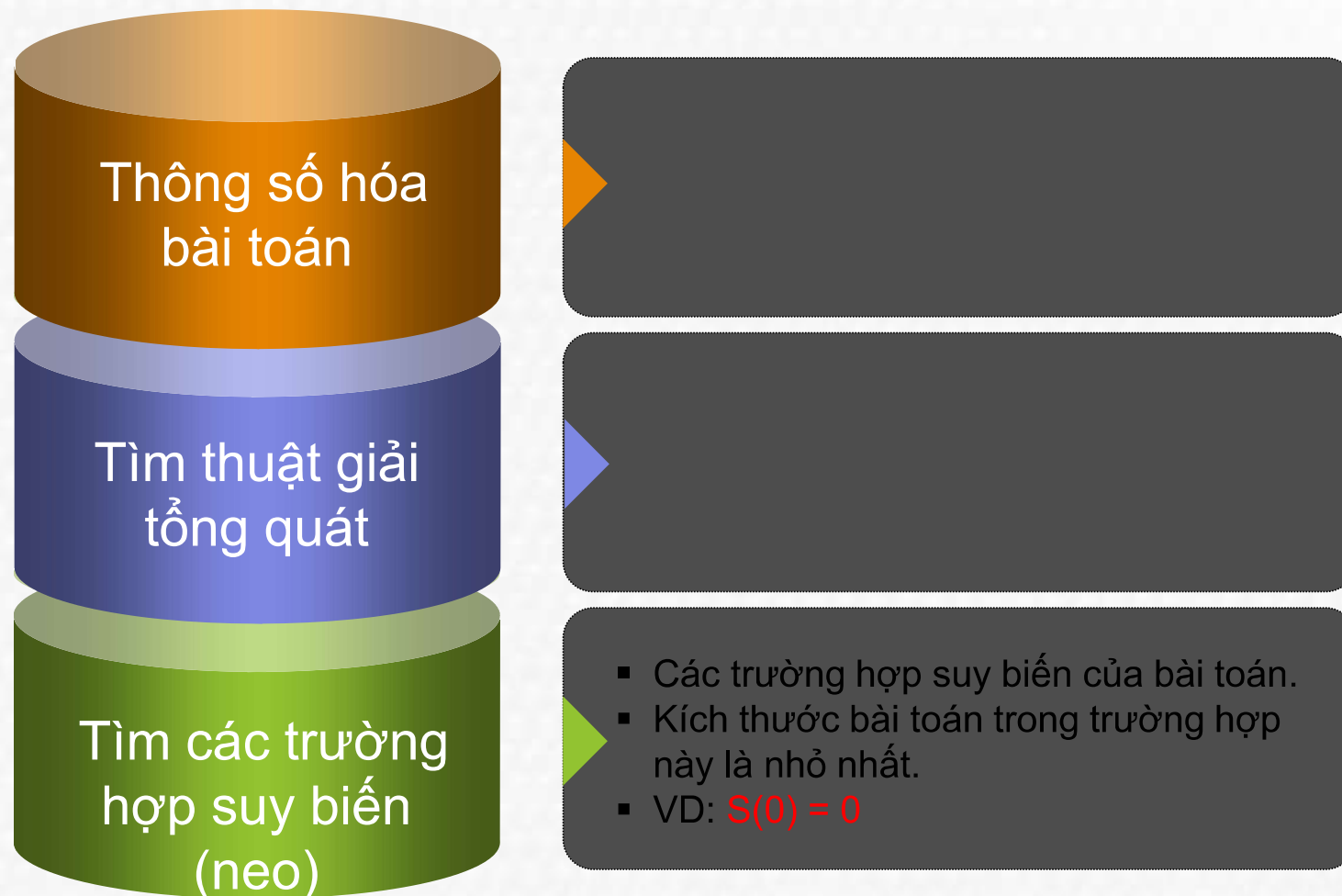


```
<Kiểu> TênHàm(<TS>) {  
    if (<ĐK dừng>) {  
        ...  
        return <Giá Trị>;  
    }  
    ... Vòng lặp {  
        ... TênHàm(<TS>); ...  
    }  
    ...  
}
```

Đệ quy phi tuyến.pptx

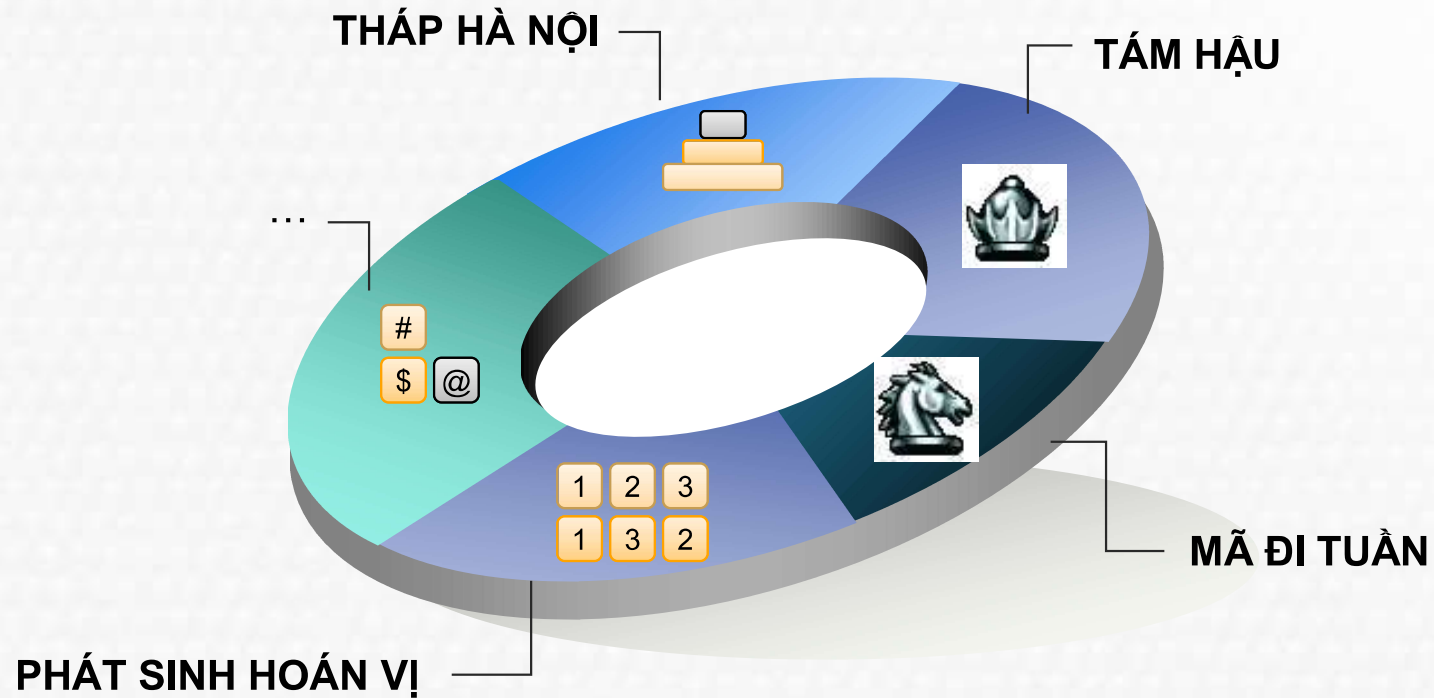
2.3. LẬP TRÌNH ĐỆ QUY

2.3.4. Các bước xây dựng hàm đệ quy



2.3. LẬP TRÌNH ĐỆ QUI

2.3.5. Một số bài toán kinh điển

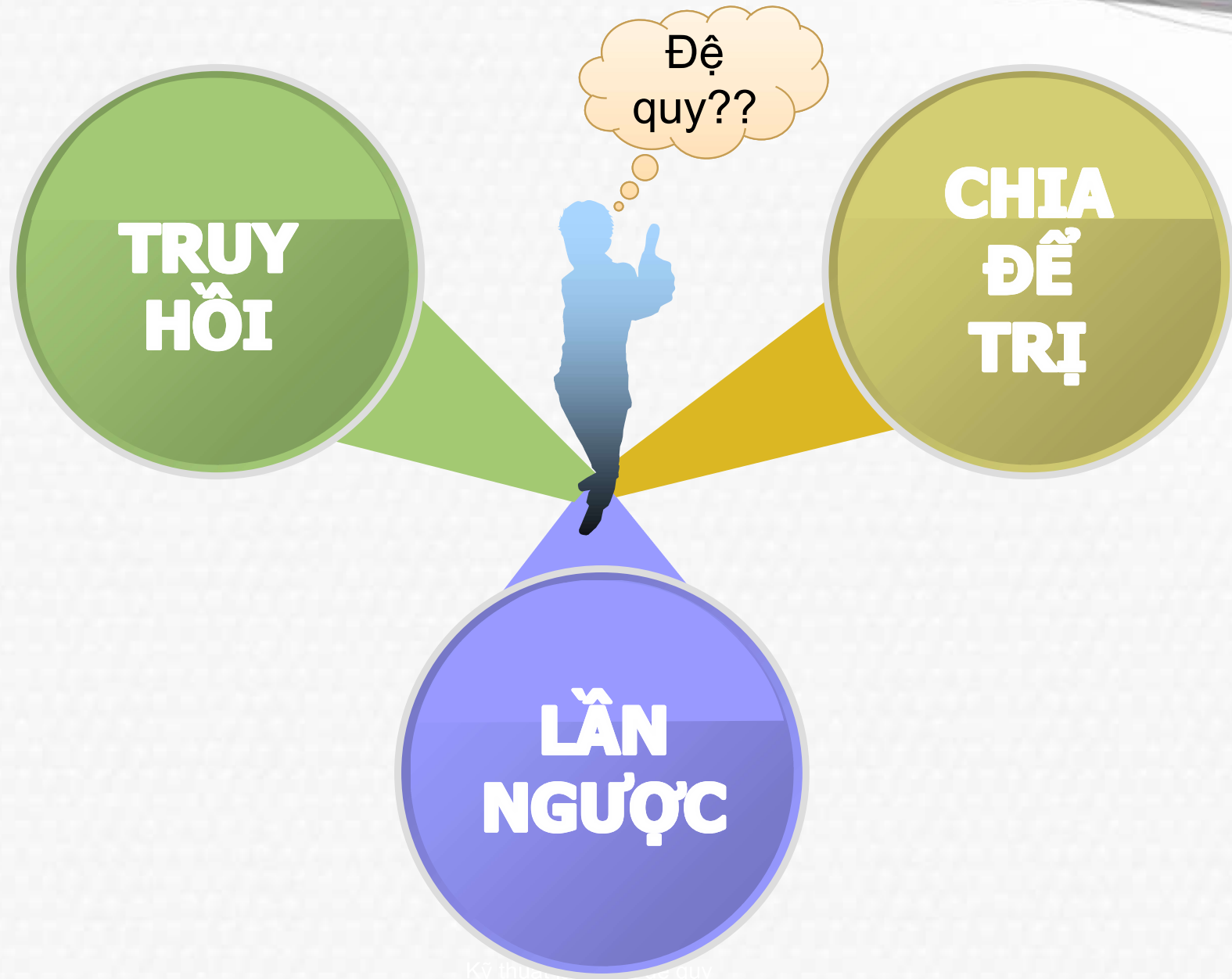


[Baitoan-Thap Ha Noi.pptx](#)

[Baitoan-Tam Hau.pptx](#)

[Baitoan-Ma đi tuan.pptx](#)

2.4. CÁC VẤN ĐỀ ĐỆ QUY THÔNG DỤNG

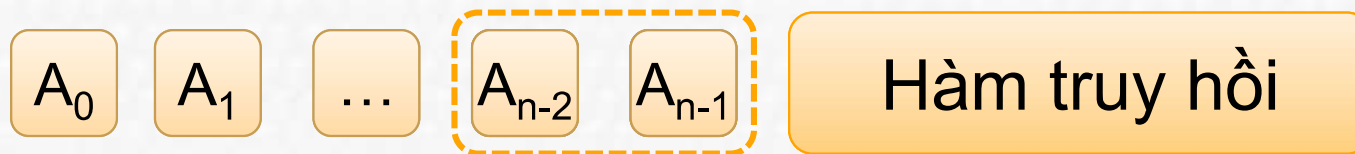
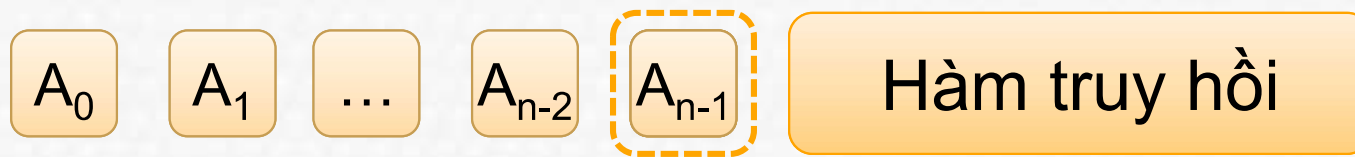


2.4. CÁC VẤN ĐỀ ĐỆ QUY THÔNG DỤNG

2.4.1. Hệ thức truy hồi

Khái niệm

- Hệ thức truy hồi của 1 dãy A_n là công thức biểu diễn phần tử A_n thông qua 1 hoặc nhiều số hạng trước của dãy.



[vidu 2-2.pptx](#)

[vidu 2-3.pptx](#)

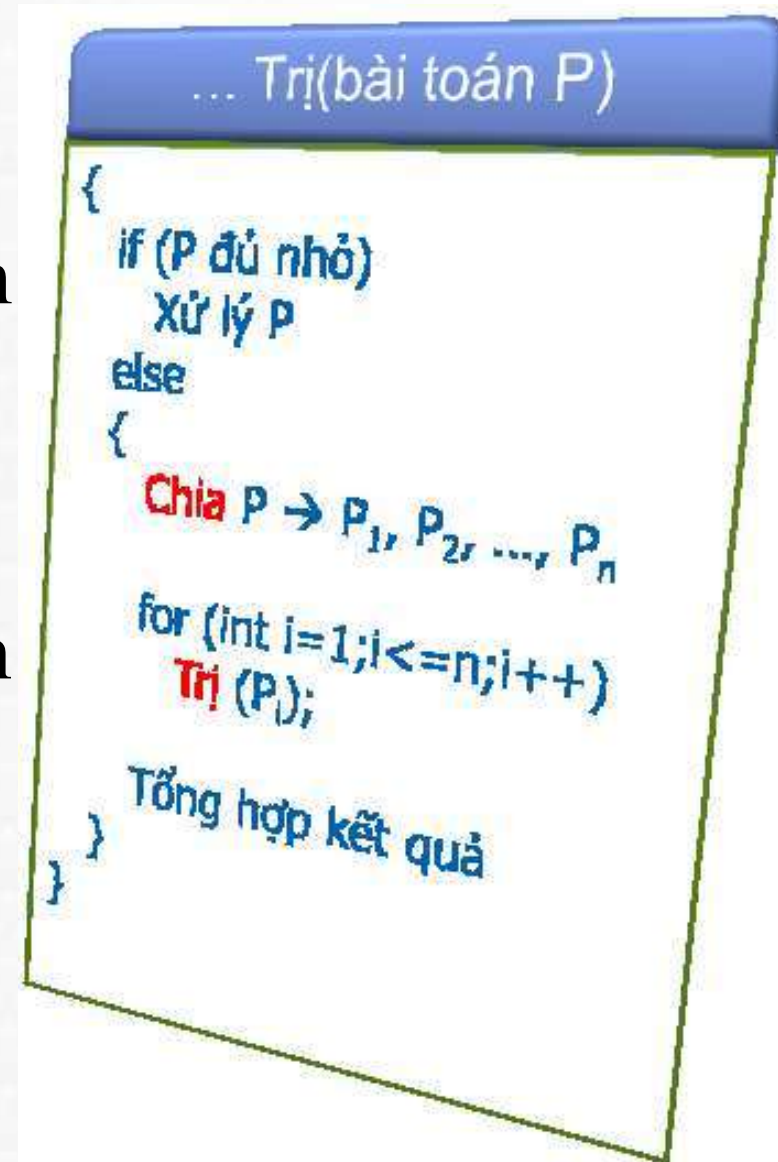
2.4. CÁC VẤN ĐỀ ĐỆ QUY THÔNG DỤNG

2.4.2. Chia để trị (divide & conquer)

Khái niệm

- Chia bài toán thành nhiều bài toán con.
- Giải quyết từng bài toán con.
- Tổng hợp kết quả từng bài toán con để ra lời giải.

[vidu 2-4.pptx](#)



2.4. CÁC VẤN ĐỀ ĐỆ QUY THÔNG DỤNG

■ Một số bài toán khác

- Bài toán tháp Hà Nội
- Các giải thuật sắp xếp: QuickSort, MergeSort
- Các giải thuật tìm kiếm trên cây nhị phân tìm kiếm, cây nhị phân nhiều nhánh tìm kiếm.

■ Lưu ý

Khi bài toán lớn được chia thành các bài toán nhỏ hơn mà những bài toán nhỏ hơn này không đơn giản nhiều so với bài toán gốc thì **không nên** dùng kỹ thuật chia để trị.

2.4. CÁC VẤN ĐỀ ĐỆ QUY THÔNG DỤNG

2.4.3. Lăn ngược (Backtracking)

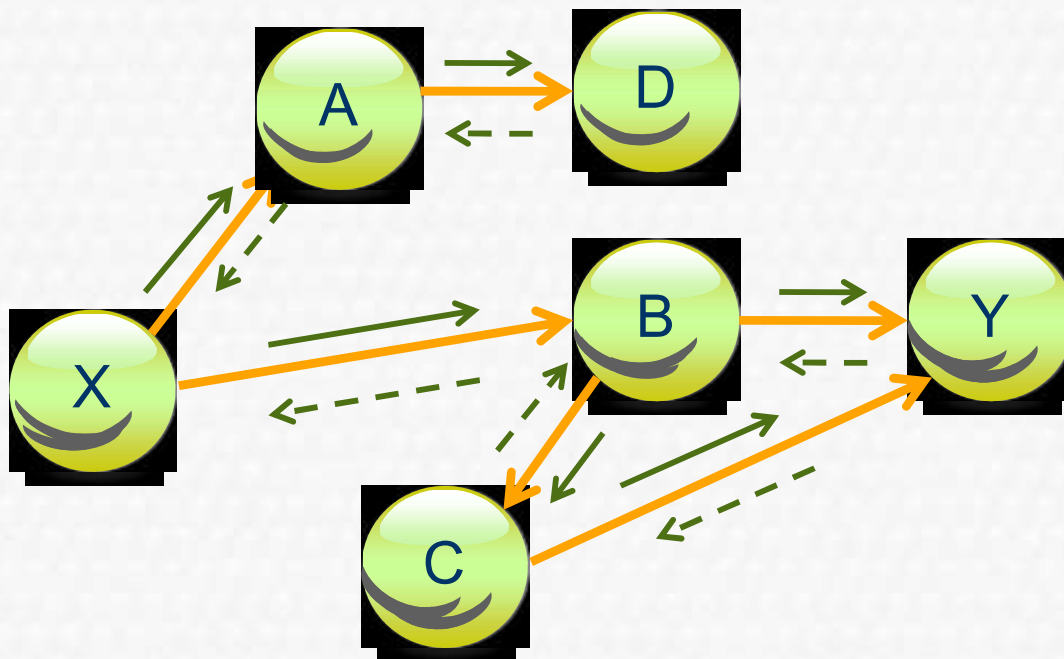
Khái niệm

- Tại bước có nhiều lựa chọn, ta chọn thử 1 bước để đi tiếp.
- Nếu không thành công thì “lăn ngược” chọn bước khác.
- Nếu đã thành công thì ghi nhận lời giải này đồng thời “lăn ngược” để truy tìm lời giải mới.
- Thích hợp giải các bài toán kinh điển như bài toán 8 Hậu và bài toán mã đi tuần.

2.4. CÁC VẤN ĐỀ ĐỆ QUY THÔNG DỤNG

Ví dụ 2.5

Tìm đường đi từ **X** đến **Y**.



2.4. CÁC VẤN ĐỀ ĐỆ QUY THÔNG DỤNG

Thuật toán Thử sai-Quay lui

void try (int i)

{ *Duyệt < không gian trạng thái ứng viên ở bước i >*

 { *ghi nhận trạng thái chấp nhận được;*

 if (i < n)

 try(i+1);

 else

Kiểm tra nghiệm – xuất kết quả;

xóa trạng thái đã ghi nhận;

}

2.5. NHẬN XÉT VỀ ĐỆ QUY

■ Ưu điểm

- Sáng sủa, dễ hiểu, nêu rõ bản chất vấn đề.
- Tiết kiệm thời gian thực hiện mã nguồn.
- Một số bài toán rất khó giải nếu không dùng đệ quy.

■ Khuyết điểm

- Tốn nhiều bộ nhớ, thời gian thực thi lâu.
- Một số tính toán có thể bị lặp lại nhiều lần.
- Một số bài toán không có lời giải đệ quy.

2.5. NHẬN XÉT VỀ ĐỆ QUY

■ Khi nào không nên sử dụng đệ quy

Nhìn chung, khi chương trình có thể viết dưới dạng lặp hoặc các cấu trúc lệnh khác thì không nên sử dụng đệ quy. Một số lý do:

- ✓ Khi một thủ tục đệ quy gọi chính nó, tập các đối tượng được sử dụng trong thủ tục này như các biến, hằng, cấu trúc ... sẽ được tạo ra.
- ✓ Việc sử dụng đệ quy đôi khi tạo ra các tính toán thừa, không cần thiết do tính chất tự động gọi thực hiện thủ tục khi chưa gặp điều kiện dừng của đệ quy.

2.5. NHẬN XÉT VỀ ĐỆ QUY

Xét bài toán tính các phần tử của dãy Fibonacci.

$$F(1) = 1, F(2) = 1,$$

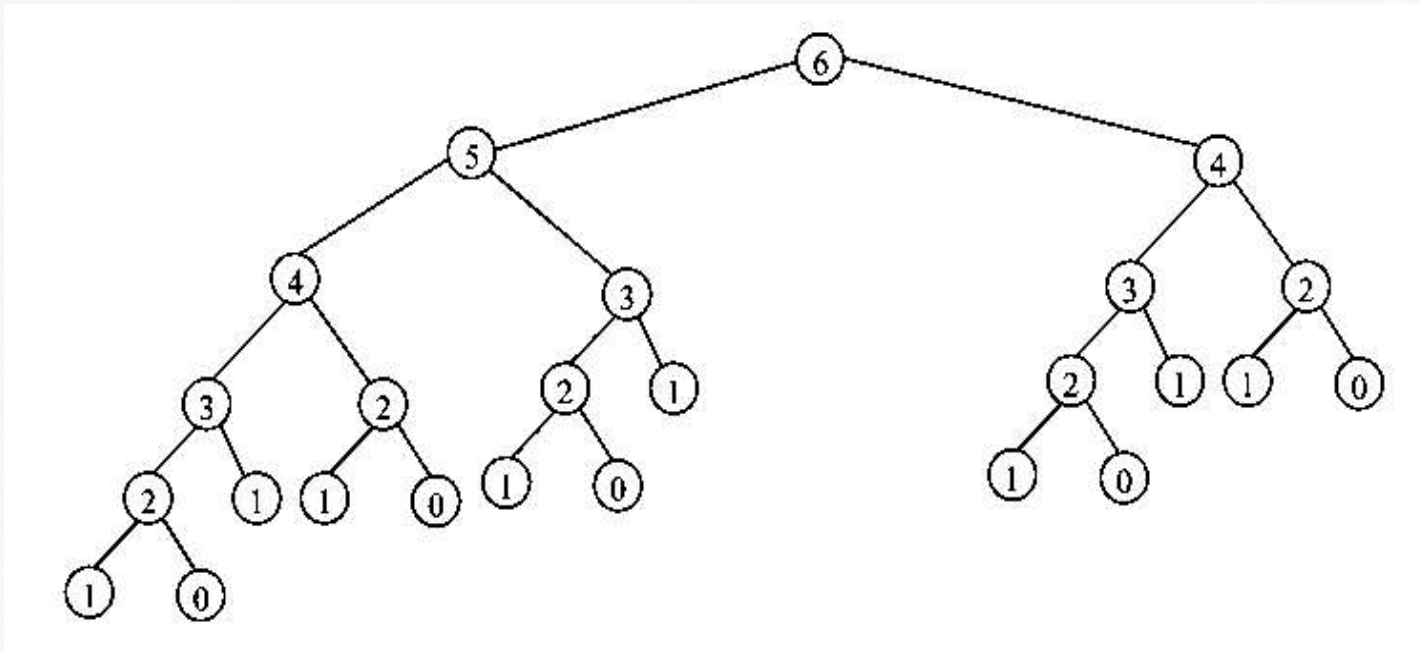
$$F(n) = F(n-1) + F(n-2), \text{ với } n > 2.$$

Rõ ràng là nhìn vào một định nghĩa đệ quy như trên, chương trình tính phần tử của dãy Fibonacci có vẻ như rất phù hợp với thuật toán đệ quy. Phương thức đệ quy để tính dãy này có thể được viết như sau:

```
int Fibonacci(int i)
{
    if ((i==1) || (i==2)) return 1;
    else return Fibonacci(i-1) + Fibonacci(i-2)
}
```

2.5. NHẬN XÉT VỀ ĐỆ QUY

Ở đây, chú ý rằng một lời gọi đệ qui $\text{Fibonacci}(n)$ sẽ dẫn tới 2 lời gọi đệ qui khác ứng với $n-1$ và $n-2$. Hai lời gọi này lại gây ra 4 lời gọi nữa .v.v, cứ như vậy số lời gọi đệ qui sẽ tăng theo cấp số mũ.



Các lời gọi đệ qui được thực hiện khi gọi thủ tục Fibonacci (6)

2.5. NHẬN XÉT VỀ ĐỆ QUY

Trong hình vẽ trên, ta thấy để tính được phần tử thứ **6** thì cần có tới **25** lời gọi! Sau đây, ta sẽ xem xét việc sử dụng vòng lặp để tính giá trị các phần tử của dãy Fibonacci như thế nào. Đầu tiên, ta khai báo một mảng F các số tự nhiên để chứa các số Fibonacci.

Vòng lặp để tính và gán các số này vào mảng rất đơn giản như sau:

$F[1]=1; F[2]=1;$

$\text{for } (i=3; i < n-1; i++) F[i] = F[i-1] + F[i-2];$

Rõ ràng là với vòng lặp này, mỗi số Fibonacci(n) chỉ được tính 1 lần thay vì được tính toán chồng chéo như ở trên. Tóm lại, nên tránh sử dụng đệ quy nếu có một giải pháp khác cho bài toán. Mặc dù vậy, một số bài toán tỏ ra rất phù hợp với phương pháp đệ quy.